

LAB 09 · FINAL CAPSTONE · ORCHESTRATION

Multi-Agent Operation

Stop operating one Ghost and start running a *system* of them — a researcher, a drafter, a reviewer, and a publisher wired into one pipeline, with handoffs, a quality gate where one Ghost checks another's work, and a supervisor that recovers when a stage fails.

DIFFICULTY · HARDEST

TIME · 2 SESSIONS, ~90 MIN EACH

SCOPE · SPANS 2 LAB SESSIONS

BUILDS ON · EVERYTHING, L1-L8

OBJECTIVES

- Decompose one real operation into **distinct roles** — researcher, drafter, reviewer, publisher — each with a single job, defined inputs and outputs, and its own guardrail.
- Build and **unit-test each role in isolation** before wiring anything, proving every stage works alone so that later failures can only live at the seams.
- Wire the **handoffs** so the output of one role becomes the clean, structured input of the next — the point where multi-agent work actually gets hard.
- Install a **quality gate**: one Ghost reviews another Ghost's output and can send it *back*, enforcing quality *between* stages, not just at the very end.
- Add a **supervisor** that detects a failed or rejected stage and recovers — retry, reroute, or escalate to a human — so the pipeline degrades safely instead of collapsing.
- Deploy the coordinated operation and write a **runbook**, then leave with the Demo Day story: what broke at a handoff, how the QA gate caught it, and how you made it recover.

Why this lab exists

Every lab so far taught you to *operate an agent*. This one teaches you to **run a system of agents** — and that is a genuine level-up, not just more of the same. A single Ghost can be careful, stateful, and guarded (that was Labs 0–8). But real operations are pipelines: someone gathers the facts, someone turns them into a draft, someone checks the draft, someone ships it. The moment you split that work across several Ghost roles, a new class of bug appears — one that does not exist in single-agent work at all.

That new bug lives **between the agents**. Each individual role can be flawless and the whole pipeline can still produce garbage, because the researcher handed the drafter something the drafter misread, or the reviewer approved on different assumptions than the drafter wrote under. In single-agent labs, if the output is wrong, you debug one prompt. Here, the output can be wrong while every prompt is right — the fault is in the *handoff*. Learning to see, test, and harden those seams is the frontier Forward Deployment Engineer skill, and it is the single most impressive thing you can put on a Demo Day stage: "I didn't build an agent, I ran a coordinated team of them, and here's how I made the team recover when one member failed."

This is why the lab spans two sessions. Session A is pure discipline — you build the roles and prove each one works *alone*, exactly the "isolate the step" reflex from Lab 0, now applied to a whole team. Session B is where it becomes a system — you wire the handoffs, add the inter-agent QA gate, deliberately break a seam, and build the recovery that makes it trustworthy. Do not try to compress the two into one sitting; the orchestration and the between-agent debugging are genuinely a session's worth of work each.

Prerequisites

- All of Labs 0–8 complete. You should be fluent in Ghost's persistent memory, its cloud workspace, deploying real web artifacts, saved workflows, permission policies, and human-in-the-loop gates.
- Ghost is live and responding in your Discord workspace, and you interact with it **entirely in Discord** — no local files, no other channels.
- Ideally, you already have three earlier builds to orchestrate: the **analyst** (Lab 6), the **growth engine** (Lab 7), and the **responder** (Lab 8). This capstone is strongest when it composes work you already trust. If you don't have all three, pick any real operation with at least three genuine stages.
- A real end-to-end operation you understand well enough to break down into roles — e.g. "research a topic → draft a briefing → review it for accuracy and policy → publish it as a live page."

BEFORE YOU START • CAPABILITY CHECK

This lab assumes you can run **more than one Ghost agent** and pass work between them — distinct agents playing distinct roles, handing off to each other. That is the ideal, and everything below is written for it.

If multi-agent is NOT accessible in your cohort, do not skip the lab — run the **single-agent fallback**. One Ghost plays each role *in turn*: it acts as the researcher, then you explicitly switch it into the drafter role, then the reviewer, then the publisher — with a **self-QA gate between every stage** where the Ghost critiques the previous stage's output before proceeding, and can send itself back to redo it. You get the exact same orchestration lesson — roles, handoffs, quality between stages, recovery — carried out by one agent wearing four hats. Throughout the lab, wherever you see "the reviewer Ghost checks the drafter Ghost," read it as "the Ghost, now in reviewer mode, checks the draft it produced in drafter mode." The **two-session structure holds either way**: Session A builds and proves each role/mode, Session B orchestrates and hardens the pipeline. Note at the top of your deliverable which mode you ran, so nobody mistakes a single-agent fallback for a multi-agent claim you can't demonstrate.

Session A · Design & build the roles

Session A is about building parts you trust, in isolation, before any of them touch each other. The entire goal of this session is to reach the end of it able to say, honestly, **"every stage works correctly on its own."** If you can say that, then any failure you hit in Session B is a handoff failure — which is exactly the bug this capstone exists to teach. Resist the urge to wire things early; the discipline is the lesson.

Map the pipeline — one job, clear I/O, and a guardrail per role

Before you build anything, define the whole team on paper. For each role name its **single job**, the exact **input** it expects, the exact **output** it produces, and the one **guardrail** it operates under. The output of each role must be shaped to be the input of the next — that alignment is what makes handoffs possible later.

```
# IN DISCORD, TO GHOST — MAP THE OPERATION BEFORE BUILDING
```

```
Help me design a multi-agent operation before we build any of it.
The operation is: research a topic, draft a briefing, review it, publish it.
```

```
For EACH role below, write me a one-row spec: JOB (one sentence),
INPUT (exactly what it receives), OUTPUT (exactly what it produces,
in a structured shape the next role can consume), GUARDRAIL (the one
rule it must never break).
```

```
Roles:
```

- RESEARCHER: gathers sources on the topic
- DRAFTER: turns research into a briefing draft
- REVIEWER: checks the draft for accuracy and policy
- PUBLISHER: deploys the approved briefing as a live page

```
Do not build anything yet. Just give me the four-row map, and flag
any place where one role's OUTPUT won't cleanly fit the next role's INPUT.
```

Read the map critically. The most important column is the seam between OUTPUT and INPUT: if the researcher emits a loose paragraph but the drafter expects a list of cited claims, you have found a future handoff bug *before* you built it. Fix the shapes now. Capture this map — it is the contract the rest of the lab is graded against.

Build each role in isolation

Now define each role as its own thing. In multi-agent mode, stand up a separate Ghost agent per role. In the single-agent fallback, define each role as a distinct **mode** the one Ghost can be switched into, saved so you can invoke it cleanly. Either way, a role knows only its own job, its input shape, its output shape, and its guardrail — nothing about the others.

```
# DEFINE ONE ROLE IN ISOLATION — THE REVIEWER
```

```
I'm defining a single role in isolation. You are the REVIEWER, and  
only the REVIEWER. You know nothing about how the draft was made.
```

```
Your JOB: given a briefing draft plus its source list, check it for  
(a) factual accuracy against the sources, (b) any claim with no source,  
(c) policy compliance (nothing unsupported stated as fact).
```

```
Your INPUT: a draft + a list of sources, pasted to you.
```

```
Your OUTPUT, always in this exact shape:
```

```
  VERDICT: APPROVE or SEND-BACK
```

```
  ISSUES: a numbered list, each tied to the specific claim
```

```
  FIX-NOTES: what the drafter must change if SEND-BACK
```

```
Your GUARDRAIL: you never rewrite the draft yourself. You only judge  
and send back. Confirm you understand, then wait for a draft.
```

Repeat for each role: give the researcher its own definition (job: gather and cite; guardrail: never fabricate a source, flag when it can't find one), the drafter its own (job: turn cited research into a draft; guardrail: never state a claim the research didn't support), and the publisher its own (job: deploy the approved briefing as a live artifact; guardrail: never publish anything not stamped APPROVE by the reviewer). Save each definition so you can invoke it identically every time — consistency between runs is what makes the pipeline reproducible.

Prove each step works ALONE before wiring — debug at the unit level

This is the payoff of Session A. Run every role by hand, in isolation, with a hand-made input, and confirm it produces a correct, correctly-shaped output. You are unit-testing the team. Do not connect anything yet.

- **Researcher alone:** give it a topic, confirm it returns real, findable sources in the agreed shape — and that it flags rather than invents when it can't find enough.
- **Drafter alone:** hand it a *hand-written* research bundle (not the researcher's real output yet), confirm it produces a draft where every claim traces to a source you gave it.
- **Reviewer alone:** hand it a draft you deliberately salted with one unsupported claim, and confirm it returns **SEND-BACK** with that exact claim in ISSUES. A reviewer that approves a bad draft is broken — fix it now, in isolation.
- **Publisher alone:** hand it an **APPROVE**-stamped briefing, confirm it deploys a live page; hand it an unstamped one, confirm it *refuses*.

Only when all four pass alone do you end Session A. Write down, per role, "passes isolation: yes." That line is your permission to start Session B — and your guarantee that anything that breaks next lives in the wiring, not the roles.

Session B · Orchestrate & harden

Session B turns four proven parts into one operation — and then does the thing that actually matters: it attacks the seams. You will wire the handoffs, install the between-stage quality gate, break a handoff on purpose to see the new class of bug, build recovery so the system degrades safely, and finally deploy it with a runbook. Budget the full session; the breaking and recovery are not optional polish, they are the point.

Wire the handoffs

Connect the roles so each one's output flows as the next one's input, in the shapes you agreed in the map. The orchestration itself is a Ghost instruction that names the sequence, the data passed at each seam, and who is in charge of moving work along.

```
# ORCHESTRATE THE PIPELINE — THE SUPERVISOR RUNS THE SEQUENCE
```

```
You are the SUPERVISOR of a four-role operation. Run this pipeline  
for the topic I give you, moving work stage to stage:
```

1. RESEARCHER → produces a cited source bundle.
2. Hand that bundle, unchanged, to the DRAFTER → produces a draft.
3. Hand the draft + the same source bundle to the REVIEWER.
4. QA GATE: if the reviewer returns SEND-BACK, route the draft and the reviewer's FIX-NOTES back to the DRAFTER for a revision, then back to the REVIEWER again. Only an APPROVE verdict may proceed.
5. Hand the APPROVED briefing to the PUBLISHER → deploys the live page.

```
At every handoff, show me exactly what you passed from one role to the  
next, so I can see the seam. Do not let any role do another role's job.  
Topic: [your topic]
```

Run it once on a clean topic and **watch the seams, not the ends**. The failure you're hunting isn't "bad final page" — it's the drafter quietly receiving something different from what the researcher emitted, or the reviewer judging against assumptions the drafter never had. Make the supervisor print each handoff payload so the seams are visible. This visibility is what lets you debug the new class of bug at all.

Add the QA gate — one Ghost reviews another's output and can send it back

The reviewer is not a rubber stamp at the end; it is a **gate between stages** with the power to reject. This is the heart of multi-agent quality: quality is enforced *during* the pipeline, by one agent judging another's work, not hoped for at the finish line. Prove the gate has real teeth by forcing a send-back.

```
# PROVE THE GATE REJECTS AND THE LOOP CLOSES
```

```
Run the pipeline again, but this time I want to see the QA gate work.
Feed the drafter a topic where the research is thin on purpose, so the
draft will contain at least one claim the sources don't support.
```

```
When the reviewer returns SEND-BACK, do NOT push it through. Show me:
```

- the reviewer's ISSUES and FIX-NOTES,
- the revised draft the drafter produced in response,
- the reviewer's second verdict.

```
Only publish if the second pass earns APPROVE. If it loops more than
twice without approval, stop and tell me — don't loop forever.
```

Confirm three things: the reviewer actually rejects (not approves-with-a-note), the drafter actually revises against the FIX-NOTES (not resubmits the same text), and the publisher never sees anything not stamped **APPROVE**. In the single-agent fallback this is the self-QA gate: the Ghost switches to reviewer mode, judges the draft it wrote in drafter mode, and sends *itself* back — the discipline is identical. A gate that can't say no is not a gate.

Break it on purpose — then make it degrade safely

Now attack the seams deliberately. In single-agent work you break one prompt; here you break a *handoff*. Do both of the following and watch what the pipeline does, then add the recovery rule each one demands.

- **Fail a handoff:** have the researcher return an empty or malformed bundle (no usable sources). Does the drafter charge ahead and hallucinate a draft from nothing, or does the pipeline notice the seam is broken and stop?
- **Feed a bad output downstream:** hand the reviewer a draft whose sources were silently swapped, so the citations no longer match. Does the reviewer catch the mismatch, or approve because the shape looked right?

```
# ADD RECOVERY SO THE SYSTEM DEGRADES SAFELY, NEVER SILENTLY
```

Add these permanent recovery rules to the supervisor:

1. **VALIDATE EACH HANDOFF.** Before passing a payload to the next role, check it is non-empty and in the expected shape. If it isn't, do NOT pass it on — stop that run and report which seam failed.
2. **A ROLE THAT CAN'T DO ITS JOB SAYS SO.** If the researcher can't find real sources, it returns "INSUFFICIENT" — it never fabricates to fill the bundle. The drafter refuses to draft from an INSUFFICIENT bundle.
3. **BOUNDED RETRY, THEN ESCALATE.** If a stage fails or the QA gate loops without approval, retry that stage at most twice. If it still fails, escalate to me in Discord with what broke and where — never push a broken payload downstream to hide the failure.

Now re-run both broken cases and show me the pipeline stopping or escalating cleanly instead of producing a confident wrong page.

The pass bar is **safe degradation**: a broken seam produces a clear stop or a human escalation, never a polished-looking wrong artifact. The most dangerous multi-agent failure is the one that looks fine — a beautiful published briefing built on an empty research bundle. If your pipeline ever ships that, the recovery rules aren't done.

Deploy & write the runbook

Make it durable and handoff-ready. Save the whole orchestration — roles, handoffs, QA gate, recovery rules — as a workflow in Ghost's own workspace, then produce a runbook a teammate could operate without you.

```
# PERSIST THE OPERATION AND DOCUMENT IT
```

```
Save this whole operation as a workflow called "briefing-pipeline",
carrying all four role definitions, the handoff shapes, the QA gate,
and the recovery rules, so any future run inherits them.
```

```
Then deploy a RUNBOOK as a live page that covers:
```

- what the operation does and each role's single job,
 - the handoff shape at every seam (what's passed, in what form),
 - how the QA gate works and what triggers a SEND-BACK,
 - every failure mode we found at the seams and how it now recovers,
 - the escalation path: when it stops and pings a human, and why.
- ```
Keep a copy of the runbook in memory too.
```

Read the runbook as the teammate who inherits it. If it explains the roles but not the *seams* — how work moves and what happens when a handoff fails — send it back. In a multi-agent operation the seams are the operation; a runbook that only documents the roles is documenting the easy half.

### FAILURE MODE — THE BUGS LIVE BETWEEN THE AGENTS

This is the whole reason the capstone is two sessions. In single-agent labs, a wrong output means a wrong prompt. Here, **every role can be individually correct and the operation can still fail**, because the fault is in the handoff — a new class of bug that only exists once you have a team. Hunt these specifically:

- **Shape drift at the seam.** The researcher emits one format, the drafter expected another; nothing errors, the draft is just quietly built on a misread. Print every handoff payload so the seam is visible.
- **Silent forward motion on a broken input.** A role receives an empty or malformed payload and confidently produces output anyway. A broken seam must stop the run, not get papered over downstream.
- **A toothless QA gate.** The reviewer "approves with notes" instead of sending back, so bad drafts sail through. If the gate can't reject and force a revision, it isn't a gate.
- **Divergent assumptions across roles.** The reviewer judges against a policy the drafter never wrote under. Roles must share the contract from the map, or they'll grade each other by different rules.
- **Unbounded loops.** Drafter and reviewer ping-pong forever with no cap. Bound the retries, then escalate to a human — a stuck pipeline that never asks for help is its own failure.
- **The confident wrong artifact.** The worst outcome: a polished, published page built on a failed handoff. It looks like success. Grade the seams, not the shine.

## DELIVERABLE

Bring the coordinated operation, the artifacts, and the Demo Day story — what broke at a handoff, how the QA gate caught it, how you made it recover.

- ☐ A one-line note at the top stating your mode: *multi-agent* (separate Ghost agents per role) or *single-agent fallback* (one Ghost switching roles with self-QA gates).
- ☐ The **pipeline map**: four roles, each with job, input, output, and guardrail, and the aligned handoff shapes between them.
- ☐ A saved "**briefing-pipeline**" **workflow** carrying the roles, handoffs, QA gate, and recovery rules.
- ☐ A successful **live run** on a real topic, ending in a deployed briefing page, with each handoff payload shown.
- ☐ A demonstrated **QA gate rejection**: a reviewer **SEND-BACK**, the drafter's revision against the fix-notes, and the eventual **APPROVE**.
- ☐ A demonstrated **handoff failure with safe recovery**: a broken seam (empty bundle or mismatched sources) that the pipeline stops or escalates instead of publishing.
- ☐ A deployed **runbook** documenting the roles, the seams, the failure modes found, and the escalation path.
- ☐ Your written **Demo Day narrative**: *what broke at the handoff, how the QA gate caught it, how I made it recover*.

## Stretch goals

- **Retry-then-escalate supervisor.** Formalize the recovery into a real supervisor that retries a failed stage a bounded number of times, then escalates to a human in Discord with a clear summary of what failed and where — the difference between a demo and an operation.
- **Pipeline metrics.** Have the supervisor track and report throughput (briefings completed per run) and the *% sent back by QA* — a visible quality signal, and a great Demo Day slide showing the gate actually working.
- **Reviewer enforces the Lab 4 policy.** Have the reviewer role apply the permission policy you wrote in Lab 4 as part of its check, so nothing gets published that would violate a guardrail — the QA gate becomes a policy gate too.
- **Compose your real fleet.** If you have them, wire the actual Lab 6 analyst, Lab 7 growth engine, and Lab 8 responder into one operation rather than a toy pipeline — the most impressive version of this capstone is your own agents cooperating.
- **Handoff schema check.** Give each seam a strict shape and have the supervisor validate against it, so a shape-drift bug is caught at the seam instead of three stages later.

## FACILITATOR NOTES

This is the course's final capstone and the strongest Demo Day piece — a student who can run a coordinated team of agents has the frontier FDE skill. Protect the two-session structure and don't let it get crammed into one.

- **Time-box the two sessions firmly.** Session A ends when every role passes in isolation — nothing wired. Make students state "passes isolation: yes" per role before they leave; a student who starts wiring in Session A will spend Session B unable to tell a role bug from a handoff bug, which defeats the whole lesson. Session B is orchestrate, break, recover, deploy — hold roughly a third of it for the breaking and recovery, because that's where the grade lives.
- **Running the single-agent fallback.** Expect a chunk of the room to lack multi-agent access — that is fine and full-credit. Coach them to make role switches *explicit and clean*: the Ghost must fully drop the drafter's context and judge as the reviewer, then send itself back. The common fallback failure is a Ghost that "reviews" its own draft with a rubber stamp because it's still in author mode. Insist on the same OUTPUT shapes (VERDICT / ISSUES / FIX-NOTES) so the self-QA gate has real teeth. The lesson — roles, handoffs, quality between stages, recovery — is identical; only the number of agents differs.
- **Grade the seams, not the ends.** The tempting submission is a beautiful published briefing. Ask to see the handoff payloads and force the two failure demos — the QA send-back and the broken-seam recovery. A pipeline that only ran clean has not demonstrated the thing this lab teaches.
- **Own a broken-seam trap.** Have an empty-research-bundle case and a swapped-sources case ready to hand out, so every student sees the pipeline face a real handoff failure rather than a soft self-authored one.
- **Watch for the confident wrong artifact.** The subtle failure is a polished page built on a failed handoff. If a student's pipeline ever publishes from an INSUFFICIENT or malformed input, that's an incomplete recovery, no matter how good the page looks.
- **Keep everything inside Ghost.** All interaction in Discord; roles, workflow, runbook, and memory in each student's own Ghost workspace; artifacts deployed from Ghost. No local files, and no claims of unattended or scheduled runs the student can't demonstrate live — the supervisor moves work when triggered.